

Mallow Technologies

Senior Laravel Developer

Mini Task — Take-Home Assignment

Thank you for taking the time to work on this assignment. Please read the full brief before you start. If anything is ambiguous, use your best judgment, make a reasonable assumption, and document it in your README — we're as interested in how you handle ambiguity as in the result.

Brief: Subscription Billing & Usage-Metering System

Design and build a small backend that meters customer usage against a subscription plan and produces billing, for a multi-tenant SaaS-style scenario.

Scope

- Merchants (tenants), each defining one or more Plans (name, base price, billing cycle, included usage units, overage rate per unit).
- Customers subscribe to a merchant's plan.
- Usage Events are recorded per customer per day (e.g. API calls) — assume high write volume.
- At cycle end, the system generates an invoice: base price + overage beyond the included allowance, with proration if the subscription started mid-cycle.

Functional Requirements

1. Design a normalized, indexed schema. Document explicitly how you'd expect it to hold up with 50L+ usage-event rows and note any denormalization/partitioning you'd consider.
2. POST /usage endpoint to record a usage event. It should be safe to call at high throughput and idempotent (a retried request must not double-count).
3. A queued, chunked job that aggregates a customer's daily usage and, at cycle end, generates the invoice with correct proration and overage math.
4. Cache plan/pricing lookups (Redis or array cache is fine for the exercise) and document your invalidation strategy.
5. GET /merchants/{id}/dashboard returning: top 5 customers by usage this month, projected overage revenue for the current cycle, and customers whose usage dropped >50% month-over-month (churn risk).
6. Basic rate-limiting on the usage endpoint.
7. Tests for the aggregation and billing calculation, including proration and overage edge cases.
8. Handle a customer upgrading or downgrading their plan mid-cycle: usage recorded before the change must be billed at the original plan's rate, and usage after the change at the new plan's rate, with proration reflecting both segments correctly.

Suggested UI Reference (Wireframe)

This is a low-fidelity layout reference for the dashboard — it shows the expected panels and data points so you can visualize scope before you start. Visual styling is entirely up to you; we're evaluating the schema, aggregation and caching decisions behind it, not the UI polish.

Current Cycle Usage

184,320 / 250,000 units

Projected Overage Revenue

₹ 42,600

Active Plan

Growth — monthly

Top 5 Customers by Usage (this cycle)

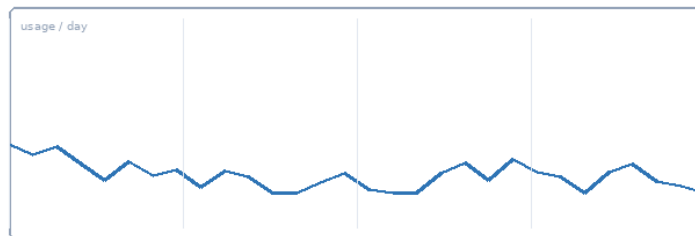
Customer	Usage	% of Allowance
Beta Retail Pvt Ltd	38,200	91%
Craft Foods Co.	31,050	78%
Delta Mart	27,900	70%

... (top 5, sorted desc.)

⚠ Churn Risk (usage ↓ >50% MoM)

- Nova Traders — 62% drop
- QuickMart — 55% drop

Daily Usage Trend (last 30 days)



System status (informational)

Plan pricing cache: Redis, TTL 10m
 Nightly aggregation job: queued,
 chunked (5k rows/batch)
 Usage endpoint: rate-limited
 120 req/min per API key

What We're Looking For

- The schema and indexing choices, and whether the reasoning scale is sounds good.
- Whether the queuing/caching choices are deliberate (chunking, idempotency, cache invalidation) rather than incidental.
- Code architecture: separation of concerns, use of services/actions/DTOs as appropriate, not everything crammed into controllers.
- A README that reads like something you'd actually hand a team — architecture summary, trade-offs made under time pressure, what you'd do differently with more time.

AI-assisted development (Copilot, Cursor, Claude, etc.) is welcome and encouraged for this task — we're as interested in how well you use these tools as in the output itself. See the Prompt Log requirement below.

What to Submit

- Codebase: a GitHub repo (private, shared with us) containing the full application.
- README.md: architecture summary, setup/run instructions, and a clear write-up of assumptions and design decisions made. If any requirement is ambiguous, make a reasonable assumption, proceed, and document it here — don't stop and wait to ask.
- Prompt Log: if you used AI-assisted tools (Copilot/Cursor/Claude etc.), include screenshots of the actual prompts you used (from your chat/IDE panel) — so we can see exactly what was asked. Save these in a /prompts folder or embed them in the README.
- Screen Recording: a short (5–10 min) video walkthrough of the working application and its architecture, narrated by you — walk us through your key decisions. A silent screen capture isn't sufficient.

Timebox & Submission

- Timebox: 3 days elapsed time from receiving this brief.
- Please share your GitHub repo link, README, prompt screenshots (if applicable) and narrated screen recording with [recruiter/HR email] once ready.

- If anything in this brief is unclear, think it through, make the call you believe is most reasonable, and document your reasoning in the README.md — we're evaluating your judgment under ambiguity, not whether you guessed our exact intent.